All SQL statements, including SELECT statements (DQL), DML, and DDL, run inside a transaction.

Transactions are normally automatic and implicit, although as the developer you may use explicit transaction blocks with the keywords BEGIN and then COMMIT or ROLLBACK. You may also lock resources explicitly.

PostgreSQL reports which lock types conflict with each other in the *Conflicting Lock Modes* table on the *Explicit Locking*[4] documentation page. Familiarize yourself with different DDL and DML statement types and which locks they take.

If you're a Rails developer who has written Migrations, you'll first need to translate the Active Record migration methods like add_column into the SQL statements like ALTER TABLE that they generate. Once you have a SQL statement, you can see what the needs are from the resource being changed, like a table or row, and whether it needs access exclusively for itself, or shared access.

With this level of understanding, you can create less disruptive modifications and SQL statements, and less lock contention from the code and SQL you write. The benefit of reducing lock contention code is that you might achieve fewer errors and higher concurrency.

If you do have lock contention and lock issues, how can you gain more insights into what's happening?

As you saw earlier, PostgreSQL system catalogs do internal bookkeeping and thus provide visibility into what's happening including for lock activity. You can query the pg_locks catalog views. Review the *Lock Monitoring*[5] PostgreSQL wiki for more information.

To get some hands on experience with lock contention, one way is to simulate it using your local database. You'll do that next!

In this exercise you'll create two transactions that contend with one another.

Follow the instructions in the SQL comments (they start with two dashes) below, opening two psql sessions connected to the Rideshare database.

sql/locks_and_blocks.sql

```
-- Start "psql1" and "psql2" in two Terminal windows
-- psql --dbname rideshare_development

-- "psql1"
BEGIN; -- start a transaction
```

4. https://www.postgresql.org/docs/current/explicit-locking.html
5. https://wiki.postgresql.org/wiki/Lock_Monitoring